

```
import os
import subprocess
html_content = """
```

# Academic Arena

## Complete End-to-End System Workflow & Technical Architecture Documentation

Comprehensive File-by-File Blueprint • Data Flow Specifications • Firebase & Express Infrastructure

### 1. Executive Summary & System Overview

**Academic Arena** is a gamified educational mobile and web platform built with React Native (Expo SDK 52, Expo Router v4), TypeScript, Node.js, Express, Firebase Auth, and Firebase Firestore database. The application enables students (Classes 6–12) to take single-player quiz runs, battle AI bots in 1v1 challenge matches, track real-time level progression, customize user profile aesthetics (avatar borders, title badges), compete on global/weekly leaderboards, earn UPI rewards, and manage content through a secure Admin Control Center.

#### Core Technology Stack

| Layer               | Technologies Used                                                              | Key Responsibilities                                                                                                                                      |
|---------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend UI         | React Native, Expo SDK 52, Expo Router v4, TypeScript, React Native Reanimated | Cross-platform UI components, screen routing, smooth animations, sound/haptic feedback.                                                                   |
| State Management    | Zustand, AsyncStorage                                                          | Client-side global state for authentication ( <code>authStore</code> ), user profile ( <code>userStore</code> ), and gameplay ( <code>gameStore</code> ). |
| API Layer           | Axios with JWT Bearer Interceptors                                             | Communicates with Express backend, attaches Firebase ID tokens automatically to headers.                                                                  |
| Backend API         | Node.js, Express.js, Firebase Admin SDK                                        | RESTful endpoints, request logging, token authentication middleware, admin guards, game balance calculations.                                             |
| Authentication & DB | Firebase Auth, Cloud Firestore                                                 | User identity management, real-time Firestore database persistence (users, runs, questions, rewards).                                                     |

## 2. End-to-End Workflow & Data Flow Architecture

### Flow 1: User Registration, Authentication & Token Synchronization

How user auth starts at login/signup, acquires Firebase JWT, syncs with backend, and initializes store.

```
[User Interface] (app/(auth)/index.tsx or signup.tsx)
  |
  ▼ (User enters credentials or taps Google Auth)
[Firebase Client SDK] (lib/firebase.ts -> signInWithEmailAndPassword / createUserWithEmailAndPas
  |
  ▼ (Validates credentials & returns Firebase User Credential + ID Token)
[Auth Store] (stores/authStore.ts -> setToken, setUser)
  |
  ▼ (Triggers token sync request: POST /api/auth/sync with Bearer Header)
[Axios API Client] (lib/api.ts -> attach Bearer token)
  |
  ▼
[Express Server] (backend/src/index.js)
  |
  ▼
[verifyToken Middleware] (backend/src/middleware/verifyToken.js)
  | (Verifies ID Token via Firebase Admin SDK backend/src/config/firebase.js)
  ▼
[Auth Route Handler] (backend/src/routes/auth.js -> /sync)
  | (Searches or creates user in Firestore via backend/src/models/User.js)
  ▼
[Firestore Database] ('users' collection)
  |
  ▼ (Returns synced user profile object: totalEXP, role, borders, badges, UPI ID)
[User Store] (stores/userStore.ts -> setProfile)
  |
  ▼
[Root Router] (app/_layout.tsx redirects from (auth) to (main)/index.tsx)
```

### Flow 2: Solo Arena Game Run Session Lifecycle

From class/subject selection, fetching question packets, live question timers, heart deductions, to run submission & EXP updates.

```

[Game Setup Screen] (app/(main)/game/index.tsx)
  | (Select Class 6-12, Subject, Packet 1-10)
  ▼ (Navigates to /game/[runId].tsx with URL params)
[Solo Game Screen] (app/(main)/game/[runId].tsx)
  |
  ▼ (Calls GET /api/questions?class=10&subject=Mathematics&packet=1&difficulty=1)
[Questions Route] (backend/src/routes/questions.js)
  | (Filters questions from backend/data/questions.js dataset / Firestore)
  ▼
[Game Store / State] (stores/gameStore.ts -> initialize 15 questions, hearts=3, score=0, timer=1)
  |
  | ──▶ (User selects option -> Check correct/wrong)
  |     |
  |     └─ Correct: Add base XP + streak multiplier + sound/haptics
  |         └─ Wrong: Lose 1 heart. If hearts == 0 -> Trigger Game Over modal
  |
  ▼ (Run Complete: User completes 15 questions or runs out of hearts)
[Axios API Client] (POST /api/runs)
  |
  ▼
[Runs Router] (backend/src/routes/runs.js)
  | 1. Calculates total EXP (score * base + heart bonus + streak bonus)
  | 2. Updates user stats in Firestore ('users' collection: totalEXP, totalAnswered, gamesP
  | 3. Creates new 'runs' document in Firestore
  ▼
[Client Response] Returns run summary -> Updates userStore totalEXP -> Displays Run Result Modal

```

### Flow 3: 1v1 Bot Challenge Arena Lifecycle

Real-time dual-simulated competition scaling bot accuracy & speed against player totalEXP.

[Challenge Screen] (app/(main)/game/challenge.tsx)

- 1. Computes Bot Difficulty (Level 1 to 10 based on player EXP)
  - Bot Accuracy:  $\text{Math.min}(0.70 + (\text{playerExp}/50000)*0.22, 0.92)$  [70% - 92%]
  - Bot Response Speed: 3000ms to 9000ms
  - Auto-advances question packet based on challengeWins
- 2. Parallel Gameplay Loop:
  - Player Timer: 15 seconds per question
  - Bot Simulation Timeout: Bot answers randomly within botSpeed range using botAcc

▼ (Match Finish: Player or Bot reaches 0 hearts / finishes 15 questions)

[Runs API Endpoint] (POST /api/runs with mode='challenge', challengeDifficulty, isChallengeWin)

▼

[Runs Router & Firestore]

| Updates user profile: challengeWins, challengeLosses, highestChallengeDifficulty

▼

[UI Result Screen] Displays Victory / Defeat card + Earned EXP & Heart Bonus EXP

#### Flow 4: Leaderboard & Rewards Workflow

Fetching weekly/total rankings, checking reward eligibility, and submitting UPI ID for payout claims.

[Leaderboard Screen] (app/(main)/leaderboard.tsx)

▼ (Calls GET /api/leaderboard?tab=weekly | total | challenge)

[Leaderboard Router] (backend/src/routes/leaderboard.js)

- weekly: Query Firestore 'users' order by weeklyEXP desc limit 50
- total: Query Firestore 'users' order by totalEXP desc limit 50
- challenge: Query Firestore 'users' order by highestChallengeDifficulty desc, challenge

▼ (Returns sorted array of leaderboard entries + active rank)

[Leaderboard UI] Displays Podium (1st, 2nd, 3rd with Glowing Profile Cards) & Ranked List

▼ (If Top 1 on Weekly Leaderboard -> Tap "Claim ₹10 UPI Reward")

[Claim Reward Modal] (Calls POST /api/leaderboard/claim-reward with upiId)

▼

[Backend Validation] Checks user rank #1, validates UPI syntax -> Creates 'rewards' doc in Fires

▼

[Firestore Database] ('rewards' collection: status='pending', upiId, userUid, amount=10)

### 3. Complete File-by-File Technical Blueprint (All 53 Files)

Below is the detailed reference of every single file in the Academic Arena repository, covering its exact location, purpose, imports/exports, state mutations, and data flows.

#### Section A: Root & Configuration Files (7 Files)

`package.json`

Root Config

**Purpose:** Primary Node.js manifest for the Expo frontend app. Defines dependencies (Expo, React Native, Axios, Zustand, Firebase Client SDK, Expo Router, Reanimated, Lucide Icons) and NPM scripts ( `start` , `android` , `ios` , `web` ).

`app.json`

Expo Config

**Purpose:** Expo application manifest. Configures app name ("Academic Arena"), slug, version, scheme ( `academic-arena` ), splash screen, icons, dark mode configuration, and Android/iOS build settings.

`eas.json`

EAS Build

**Purpose:** Expo Application Services (EAS) build configuration for producing Android APKs/AABs and iOS builds under development, preview, and production profiles.

`tsconfig.json`

TypeScript

**Purpose:** TypeScript compiler options extending `expo/tsconfig.base` . Sets strict type checking, module resolution, and path aliases ( `@/*` ).

`.env`

Env Vars

**Purpose:** Stores environment variables for the frontend application, including backend API URL ( `EXPO_PUBLIC_API_URL` ) and Firebase web config credentials.

**Purpose:** AI developer guidelines and project documentation rules (e.g. strict Expo v57+ versioned documentation rules).

## Section B: Frontend Application & Routing (`app/`) (15 Files)

### app/index.tsx

Root Entry

**Purpose:** Root entry route. Performs an immediate redirect ( `<Redirect href="/(auth)" />` ) or to `/(main)` depending on auth state.

### app/\_layout.tsx

Root Navigation

**Purpose:** Root Expo Router layout component. Initializes custom fonts, sets up Firebase Auth listener (`onAuthStateChanged`), synchronizes state with ``authStore`` and ``userStore``, manages splash screen hiding, and wraps app with Dark Theme Provider.

### app/(auth)/\_layout.tsx

Auth Layout

**Purpose:** Layout stack for authentication screens (``index.tsx`` and ``signup.tsx``) with hidden navigation headers.

### app/(auth)/index.tsx

Login Screen

**Purpose:** Login UI screen. Handles email/password authentication via Firebase Client SDK, triggers token sync with backend ``/api/auth/sync``, updates ``authStore`` & ``userStore``, and handles error messages.

### app/(auth)/signup.tsx

Signup Screen

**Purpose:** Registration UI screen. Collects full name, email, password, class selection (6-12), creates Firebase Auth account, calls backend registration API, and initializes default user profile.

`app/(main)/_layout.tsx`

Main Tab Layout

**Purpose:** Bottom tab bar navigation for main application (Dashboard, Game Arena, Leaderboard, Profile, Admin). Renders custom headers with live user EXP, level badge, and coin count.

`app/(main)/index.tsx`

Dashboard Screen

**Purpose:** Main player home dashboard. Displays level progression bar, daily streak indicator, quick action play buttons (Solo Arena, 1v1 Bot Challenge), recent match summary, and announcements.

`app/(main)/leaderboard.tsx`

Leaderboard Screen

**Purpose:** Displays competitive rankings across 3 tabs: Weekly Arena, All-Time EXP, and 1v1 Challenge Masters. Features top 3 glowing podium cards, rank list, and ₹10 UPI reward claim modal.

`app/(main)/profile.tsx`

Profile Screen

**Purpose:** User customization center. Allows switching avatar borders (Default, Neon Cyan, Gold Crown, Fire Flame, Cyber Shield), title badges, updating UPI ID, viewing lifetime stats, and logging out.

`app/(main)/admin.tsx`

Admin Control Center

**Purpose:** Admin-only control panel. Protected by `verifyAdmin``. Features system stats cards, question bank manager (Add/Edit/Delete questions), user role manager (Grant Admin, Grant EXP), and pending reward claim approvals.

`app/(main)/help.tsx`

Help & Rules

**Purpose:** Comprehensive help manual explaining scoring rules, streak multipliers, heart bonuses, bot difficulty levels, and weekly UPI rewards.

`app/(main)/game/_layout.tsx`

Game Layout

**Purpose:** Stack layout wrapper for game sub-routes (``index``, ``[runId]``, ``challenge``).

`app/(main)/game/index.tsx`

Game Setup

**Purpose:** Match creation screen. User selects Class (6 to 12), Subject (Mathematics, Science, English, Social Science), Packet (1 to 10), and launches Solo Arena or Challenge Mode.

`app/(main)/game/[runId].tsx`

Solo Arena Game

**Purpose:** Core single-player gameplay engine. Fetches 15 questions from backend, manages 15s per-question timer, 3 hearts, streak multipliers, sound effects, question transitions, and submits run results to `/api/runs``.

`app/(main)/game/challenge.tsx`

1v1 Bot Challenge Game

**Purpose:** 1v1 Bot Challenge game engine. Dynamically scales bot difficulty (Level 1-10), bot accuracy (70%-92%), and bot speed (3s-9s). Simulates live bot opponent, tracks dual scores, auto-advances question packets, and saves match results.



## Section C: UI Components (`components/`) (2 Files)

[components/BouncyButton.tsx](#)

Component

**Purpose:** Reusable tactile button component using `React Native Reanimated`. Provides spring bounce animation on press, sound triggers, loading spinner state, and custom styling.

[components/GlowingProfileCard.tsx](#)

Component

**Purpose:** Animated profile badge component with glowing neon borders, customizable avatar frames (Neon, Gold, Fire, Cyber), rank badge tags, and level indicators used in Leaderboards and User Profile screens.

## Section D: Libraries, Stores & Helpers (`lib/`, `stores/`, `constants/`, `types/`) (9 Files)

[lib/api.ts](#)

Axios HTTP

**Purpose:** Configured Axios instance pointing to `EXPO\_PUBLIC\_API\_URL`. Includes an request interceptor that fetches the current Firebase auth token (`auth().currentUser.getIdToken()`) and attaches it as `Authorization: Bearer`.

[lib/firebase.ts](#)

Firebase Client

**Purpose:** Initializes client-side Firebase App and exports Firebase Auth instance (`getAuth()`). Configures persistence using AsyncStorage for React Native.

[stores/authStore.ts](#)

Zustand Store

**Purpose:** Global auth state store holding `user` object, `token` string, `isAuthenticated` boolean, and actions (`setAuth`, `clearAuth`, `setToken`).

[stores/userStore.ts](#)

Zustand Store

**Purpose:** Player profile store holding `profile` data (EXP, level, streak, stats, unlocked items, UPI ID) and actions (`setProfile`, `updateStats`, `clearProfile`).

[stores/gameStore.ts](#)

Zustand Store

**Purpose:** Active gameplay state manager holding current questions array, question index, hearts remaining, active score, current streak, timer countdown, and reset actions.

[constants/config.ts](#)

Config

**Purpose:** Global application constants including default API fallback URLs, packet question limits (15 questions/run), and timing constants.

[constants/theme.ts](#)

Theme

**Purpose:** Centralized design system color palette (Dark theme backgrounds, Cyan neon accents, Gold rank colors, Card surfaces) and typography styles.

[types/index.ts](#)

TypeScript Types

**Purpose:** Core TypeScript interface definitions: `UserProfile`, `Question`, `GameRun`, `LeaderboardEntry`, `RewardClaim`, `ApiResponse`.

## Section E: Backend Server Architecture (`backend/`) (20 Files)

[backend/package.json](#)

Backend Config

**Purpose:** Express server dependencies: `express`, `cors`, `dotenv`, `firebase-admin`, `express-validator`. Defines server startup scripts (`npm start`, `npm run dev`).

[backend/.env](#)

Backend Env

**Purpose:** Server environment variables including `PORT` (5000), `FIREBASE\_PROJECT\_ID`, `FIREBASE\_CLIENT\_EMAIL`, `FIREBASE\_PRIVATE\_KEY`.

[backend/src/index.js](#)

Express Entry

**Purpose:** Express app initialization. Attaches `cors()`, `express.json()`, custom logger middleware, mounts sub-routers (`/api/auth`, `/api/questions`, `/api/runs`, `/api/leaderboard`, `/api/admin`), and attaches global error handler.

[backend/src/config/db.js](#)

DB Config

**Purpose:** Database connection placeholder / status verification export for database integration.

[backend/src/config/firebase.js](#)

Firebase Admin

**Purpose:** Initializes Firebase Admin SDK using service account credentials from `.env`. Exports Admin Auth (`admin.auth()`) and Firestore Instance (`admin.firestore()`).

[backend/src/config/game.js](#)

Game Balance

**Purpose:** Master game scoring parameters: Base XP per question (100), Streak multiplier step (0.1x), Heart bonus XP (50 XP/heart), XP per Level formula (5000 XP/level).

[backend/src/middleware/logger.js](#)

Middleware

**Purpose:** HTTP request logging middleware. Logs method, URL route, status code, and response time in milliseconds for every API request.

[backend/src/middleware/errorHandler.js](#)

Middleware

**Purpose:** Global Express error handling middleware. Catches unhandled errors, formats JSON error response with status code and error messages.

[backend/src/middleware/verifyToken.js](#)

Middleware

**Purpose:** Authentication guard. Extracts Bearer token from `Authorization` header, verifies it via `admin.auth().verifyIdToken(token)`, and attaches decoded user token (`req.user`) to request object.

[backend/src/middleware/verifyAdmin.js](#)

Middleware

**Purpose:** Authorization guard for admin routes. Runs after `verifyToken`, fetches user document from Firestore (`users` collection), checks if `role === 'admin'`, and returns 403 Forbidden if unauthorized.

[backend/src/models/User.js](#)

Model

**Purpose:** User data model helper for Firestore. Encapsulates profile formatting, totalEXP to Level conversion, level calculation, and default field initialization (badges, activeBorder, challengeWins).

[backend/src/models/Run.js](#)

Model

**Purpose:** Run data model helper. Formats and validates game run records before persisting into Firestore `runs` collection.

[backend/src/models/Question.js](#)

Model

**Purpose:** Question data model helper. Formats question schema (id, class, subject, packet, difficulty, text, options, correctIndex, explanation).

[backend/src/models/Reward.js](#)

Model

**Purpose:** Reward claim model helper. Structure for UPI reward claims in Firestore `rewards` collection (id, uid, userName, upild, amount, status: 'pending'|'approved').

[backend/src/routes/auth.js](#)

Auth API

**Purpose:** Auth REST router. Endpoints: `POST /sync` (Verifies token & returns/creates Firestore user profile), `PUT /profile` (Updates avatar, border frame, title badge, UPI ID).

[backend/src/routes/questions.js](#)

Questions API

**Purpose:** Question delivery endpoint (`GET /api/questions`). Filters static question database (`backend/data/questions.js`) and Firestore question store by class, subject, packet number, and difficulty, returning 15 randomized questions.

[backend/src/routes/runs.js](#)

Runs API

**Purpose:** Game submission endpoint (`POST /api/runs`). Validates score, computes EXP earned with heart/streak bonuses, updates user document in Firestore (`totalEXP`, `gamesPlayed`, `challengeWins`), and stores run record in `runs` collection.

[backend/src/routes/leaderboard.js](#)

Leaderboard API

**Purpose:** Leaderboard endpoints (`GET /api/leaderboard?tab=...`, `POST /claim-reward`). Queries Firestore users collection sorted by EXP or challenge wins, and processes UPI reward claim requests for rank #1.

[backend/src/routes/admin.js](#)

Admin API

**Purpose:** Protected admin endpoints (`/api/admin/\*`). Handlers for fetching dashboard statistics, adding/editing/deleting questions, granting user EXP, granting admin privileges, and approving UPI reward payouts.

[backend/data/questions.js](#)

Question Dataset

**Purpose:** Extensive pre-built offline static question database containing over 6,000 curriculum questions categorized by Class (6 to 12), Subject, Packet Number, and Difficulty Rating (1 to 10).

[backend/scripts/seed.js](#)

CLI Script

**Purpose:** Database populator script. Uploads static questions from `questions.js` into Cloud Firestore `questions` collection in bulk batches.

[backend/scripts/makeAdmin.js](#)

CLI Script

**Purpose:** CLI utility script to grant administrative privileges (`role: 'admin'`) to any specified user UID in Firestore.

## 4. Cloud Firestore Database Schemas

---

### User Collection Schema (`users/{uid}`)

```
{
  "uid": "String (Firebase Auth UID)",
  "email": "String",
  "name": "String",
  "class": "Number (6 - 12)",
  "role": "String ('user' | 'admin')",
  "totalEXP": "Number",
  "weeklyEXP": "Number",
  "gamesPlayed": "Number",
  "totalAnswered": "Number",
  "totalCorrect": "Number",
  "challengeWins": "Number",
  "challengeLosses": "Number",
  "challengeGamesPlayed": "Number",
  "highestChallengeDifficulty": "Number (1 - 10)",
  "activeBorder": "String ('default' | 'neon' | 'gold' | 'fire' | 'cyber')",
  "badges": "Array<String>",
  "upiId": "String",
  "createdAt": "Timestamp",
  "updatedAt": "Timestamp"
}
```

### Game Run Collection Schema (`runs/{runId}`)

```
{
  "runId": "String",
  "userId": "String (UID)",
  "class": "Number",
  "subject": "String",
  "packet": "Number",
  "score": "Number",
  "correctAnswers": "Number",
  "questionsAnswered": "Number",
  "expEarned": "Number",
  "heartsRemaining": "Number",
  "mode": "String ('solo' | 'challenge')",
  "challengeDifficulty": "Number",
  "isChallengeWin": "Boolean",
  "timestamp": "Timestamp"
}
```

## Reward Claim Schema (`rewards/{rewardId}`)

```
{
  "id": "String",
  "userId": "String (UID)",
  "userName": "String",
  "upiId": "String",
  "amount": "Number (10)",
  "status": "String ('pending' | 'approved' | 'rejected')",
  "claimedAt": "Timestamp"
}
```



## 5. Security, Validation & Error Handling Pipeline

---

- **Authentication Guard:** All protected API requests pass through `verifyToken.js` which cryptographically validates the Firebase ID token via `admin.auth().verifyIdToken()`.
- **Role Authorization Guard:** Sensitive routes (`/api/admin/*`) execute `verifyAdmin.js` to ensure the requester possesses `role === 'admin'` in Firestore.
- **Input Sanitization:** User profile updates (UPI ID, name, class) are sanitized to prevent injection attacks and invalid data types.
- **Centralized Logging & Error Diagnostics:** The custom `logger.js` records request metrics, while `errorHandler.js` intercepts unhandled exceptions to return standard JSON error structures without leaking sensitive stack traces in production.

Generated automatically for **Academic Arena** • Full End-to-End System Documentation Artifact

```
""" with open("Academic_Arena_Documentation.html", "w", encoding="utf-8") as f: f.write(html_content)
print("HTML Documentation created successfully.")
```